HOW WEB PROGRAMMING IS MORE THAN A SERVER AND SOME CLIENTS

Laure Philips

Cracked Earth, Abraham Lake, photo by Mike Isaak Photography

About me

PhD student

I am a PhD-student at the Software Languages Lab, which is part of the Computer Science Department of the Faculty of Sciences at the Vrije Universiteit Brussel.

I am funded by a doctoral scholarship from the Flemish Institute for the Improvement of the Scientifical-Technological Research in the Industry (IWT).

WEB PROGRAMMING

HTML AGE

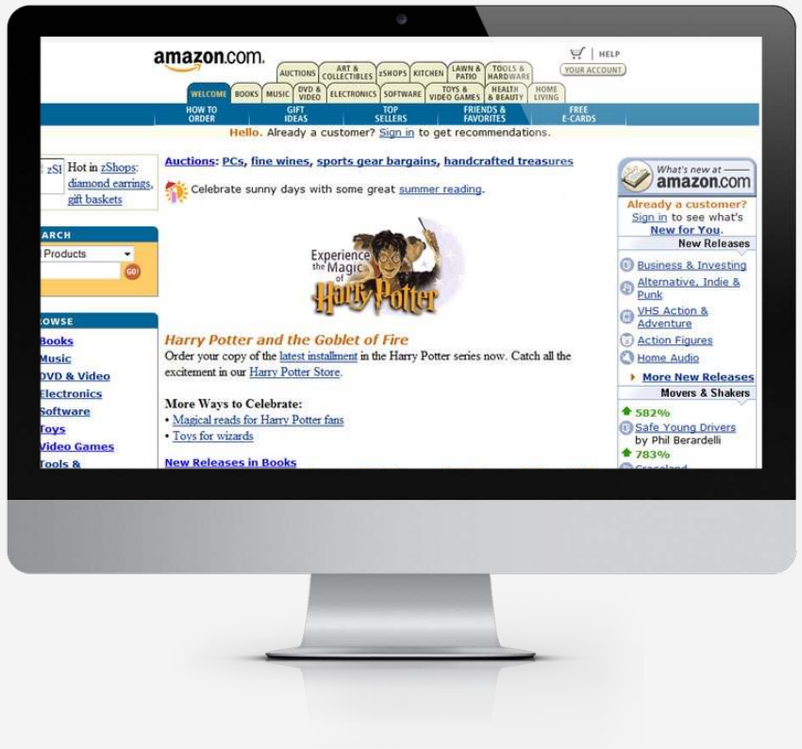
90's

STATIC

LOGIC ON THE CLIENT

NUMBER OF PAGES

WEB PROGRAMMING



LAMP AGE

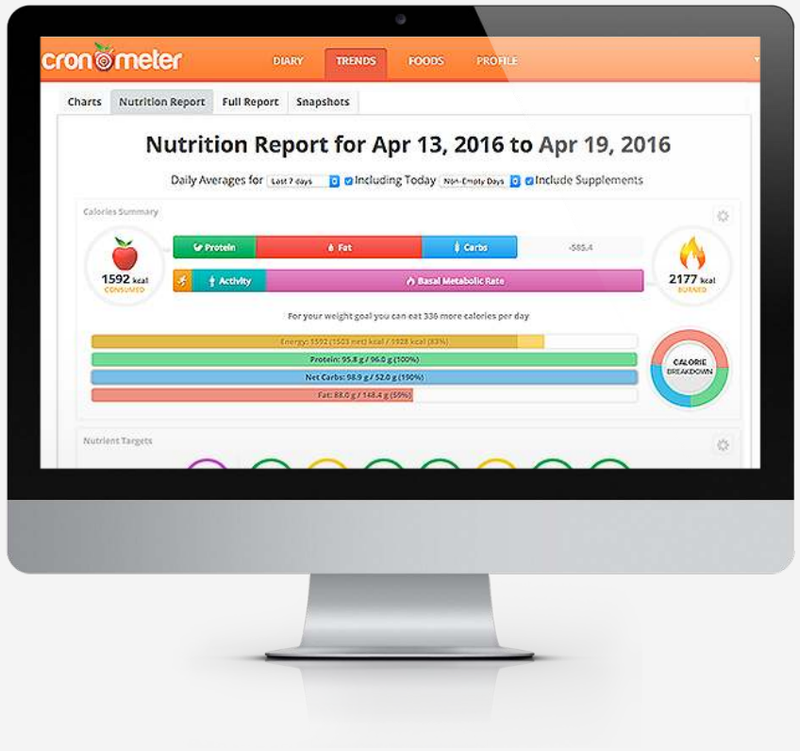
00's

STATIC

LOGIC ON THE CLIENT

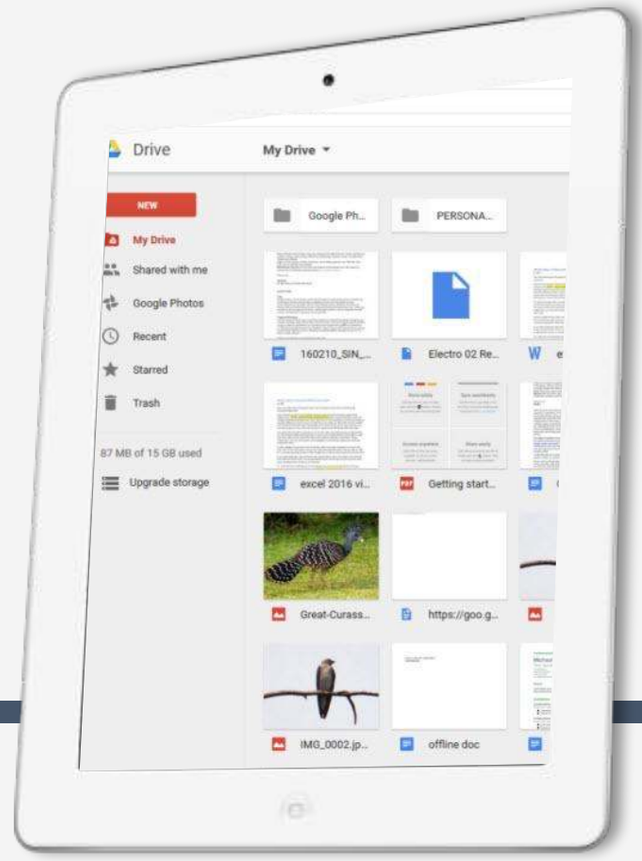
NUMBER OF PAGES

WEB PROGRAMMING



JAVASCRIPT AGE

10's



STATIC

LOGIC ON THE CLIENT

NUMBER OF PAGES

RICH INTERNET APPLICATIONS

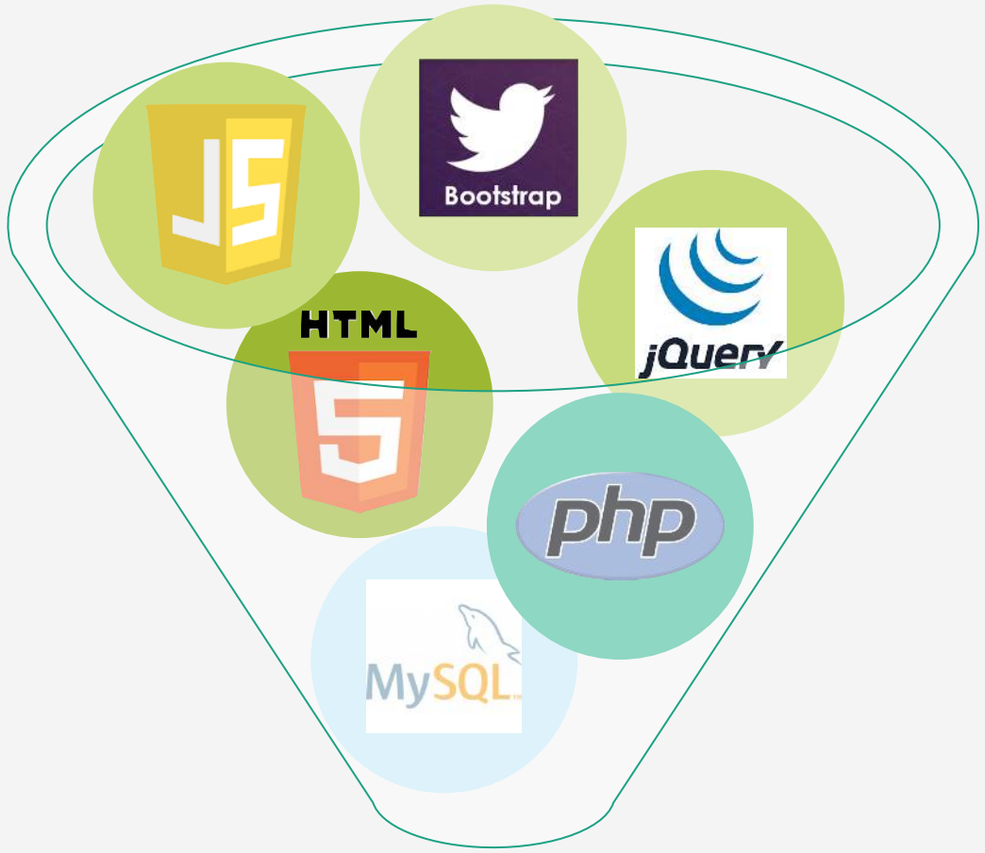
R ich client with parts of the application logic

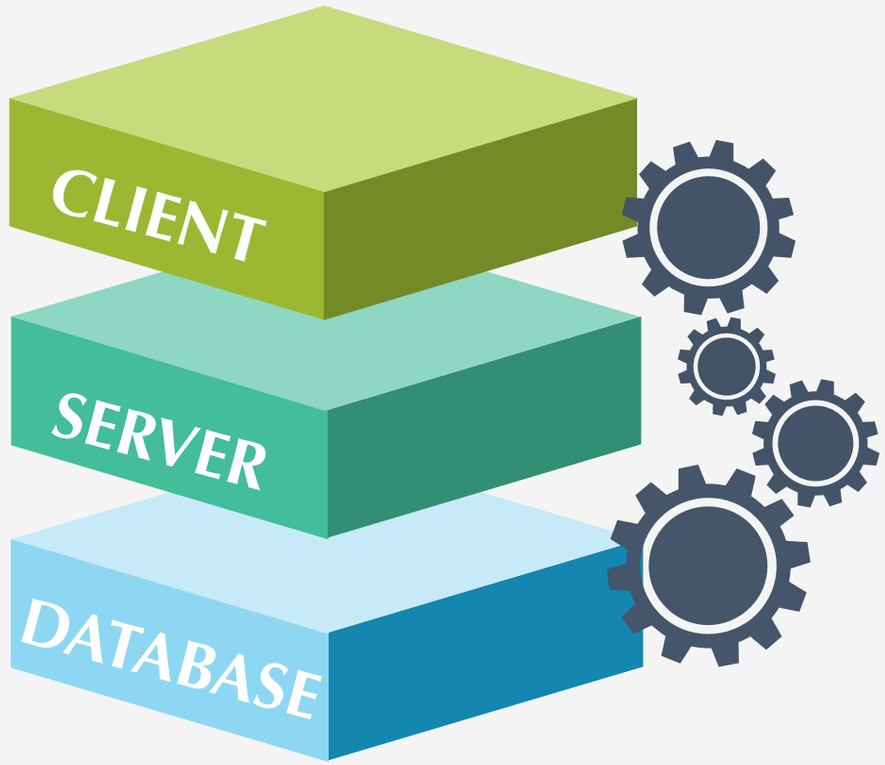
I page apps with reactive content

O ffline availability of data on the client side

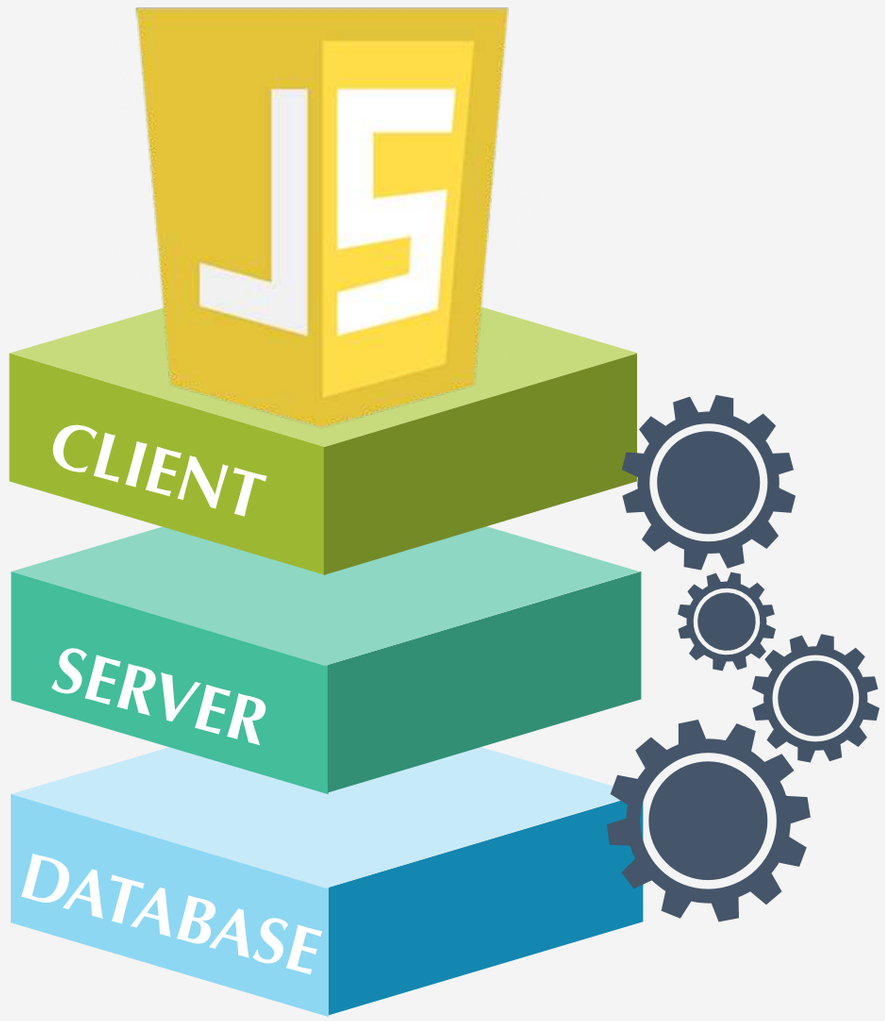
S ervices from other servers in a mash-up

THREE TIER MODEL





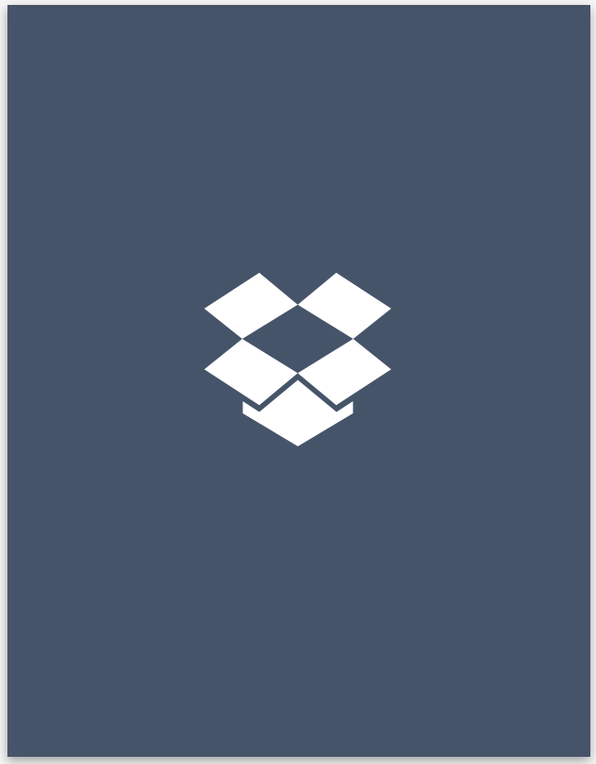
THREE TIER MODEL



Tierless Programming

Tierless (also called single-tier, multi-tier or isomorphic) programming enables developing a web application as a single mono-linguistic application, which renders its development akin to that of a desktop application.

TIERLESS APPROACHES



FRAMEWORKS

REWRITING TOOLS

LANGUAGES

TIERLESS FRAMEWORKS

GWT

Java programming language on the server and on the client.

METEOR

JavaScript programming language on the server and on the client.

HELLO WORLD (CHAT)

```
1 Messages = new Meteor.Collection("messages");
2
3 if (Meteor.isClient) {
4
5     Template.input.events({
6         'click .sendMsg' : function (el) {
7             Messages.insert({msg : el.value, date: new Date()})
8         }
9     })
10 }
11
12 if (Meteor.isServer) {
13     Messages.deny({
14         insert: function (userId, message) {
15             return message.msg.length > 0
16         }
17     })
18 }
```

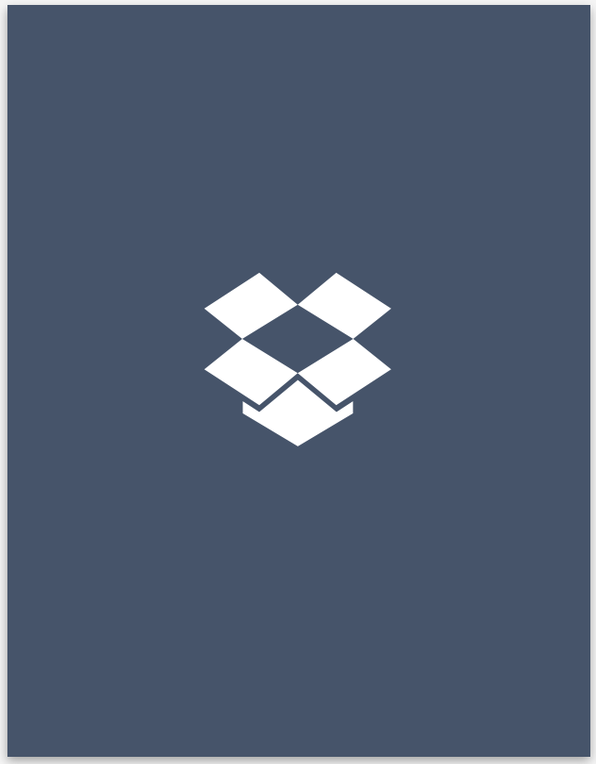
METEOR

HELLO WORLD (CHAT)

```
1 <template name="messages">
2   <h3>Messages:</h3>
3   <div id="msgs">
4     {{#each messages}}
5       <p>{{msg}}</p>
6     {{/each}}
7   </div>
8
9   <form>
10    <input type="text" name="text" placeholder="Type here"/>
11  </form>
12
13 </template>
```

METEOR

TIERLESS APPROACHES



FRAMEWORKS

- One mainstream language
- Extensive set of libraries
- Reuse JavaScript libraries

TIERLESS REWRITING TOOLS

J-ORCHESTRA

Java bytecode rewriting of desktop app to distributed app based on a distribution plan

RIPLEY

based on Volta, focusing on safety by replicated execution

VOLTA

Splits annotated .net code to server and client js code

HELLO WORLD (FIBONACCI)

```
1 [RunAt("Server")]
2 public class Main {
3     public int Fibonacci(int i) {
4         if (n < 2)
5             return n
6         else
7             return Fibonacci(n - 1) +
8                    Fibonacci(n - 2);
9     }
10 }
```

```
1 [Async]
2 public extern void Fibonacci(int n, Action<int> continuation);
3
4 mc.Fibonacci(argument, (res) =>
5     Console.WriteLine("Result = {0}", res));
```

TIERLESS APPROACHES

REWRITING TOOLS

- Mainstream language
- Developer annotations
- Harder to integrate JavaScript libraries

TIERLESS LANGUAGES

LINKS

functional language with JavaScript-like syntax with client/server keywords

UR/WEB

typed, functional language with a focus on safety

HOP

superset of JavaScript with explicit tier switching markers ~ en $.

OPA

typed language with semi-automatic distribution and JavaScript-like syntax

HELLO WORLD (CHAT)

```
1 type message = {
2   string msg  /** Content entered by the user */
3 }
4
5 module Model {
6   private Network.network(message) room = Network.cloud("room")
7
8   exposed function broadcast(message) {
9     Network.broadcast(message, room);
10  }
11
12  function register_message_callback(callback) {
13    Network.add_callback(callback, room);
14  }
15 }
```

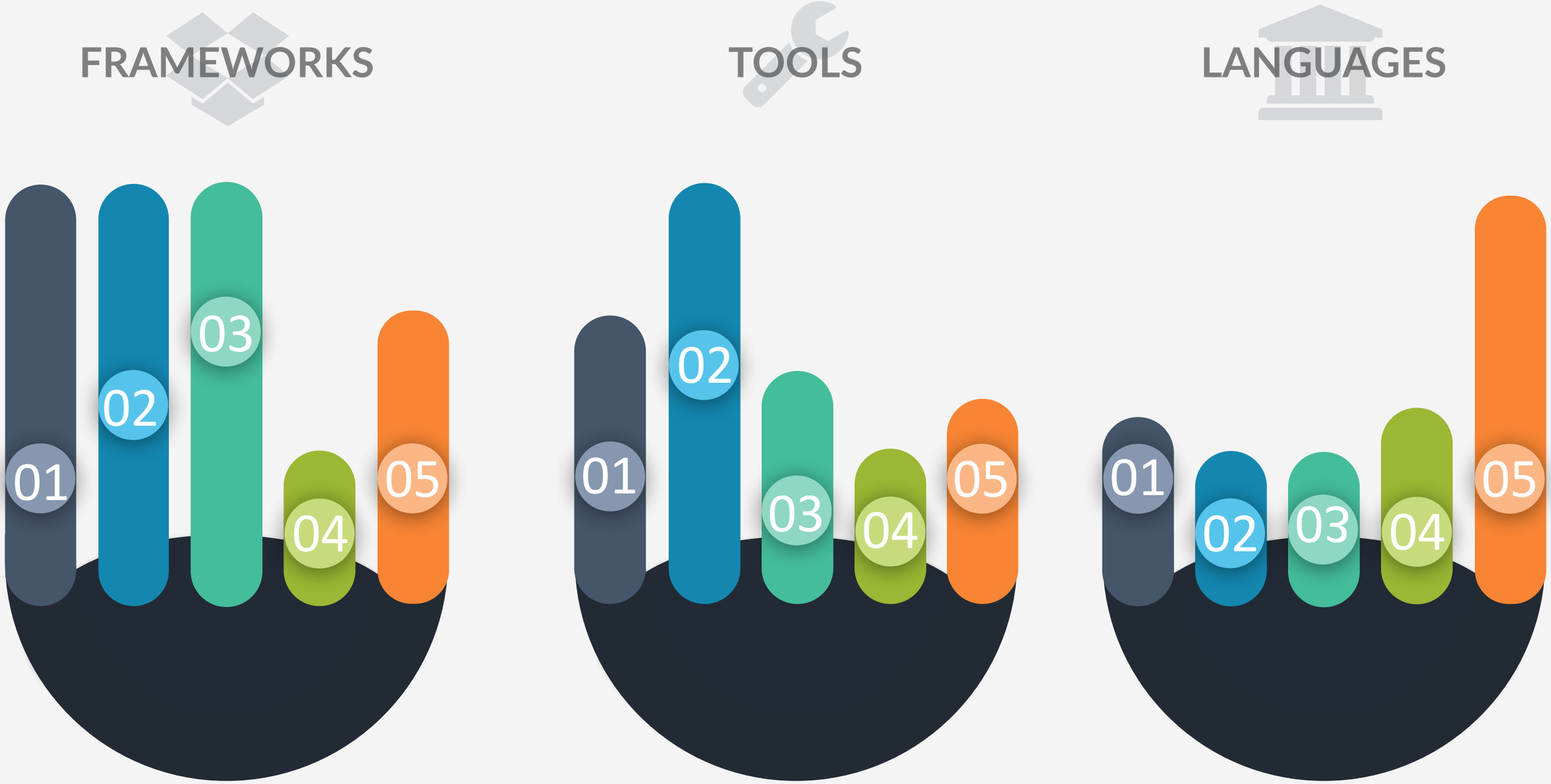
HELLO WORLD (CHAT)

```
1 function chat_html() {
2   <div id=#conversation
3     onready={function(_) {
4       Model.register_message_callback(user_update)}} />
5   <div>
6     <input id=#entry  type=text />
7     <button type=button onclick={function(_) { broadcast() }}>Post</>
8   </div>
9 }
10
11 function broadcast() {
12   Model.broadcast({msg: Dom.get_value(#entry)});
13 }
14
15 function user_update(message msg) {
16   line = <p> {msg.msg} </p>
17   #conversation =+ line;
18 }
```

TIERLESS APPROACHES

LANGUAGES

- New language
- Focus on particular concern
- Integrated UI and DB templating



01 Tool support

02 Mainstream language

03 Full-fledged JS on the client

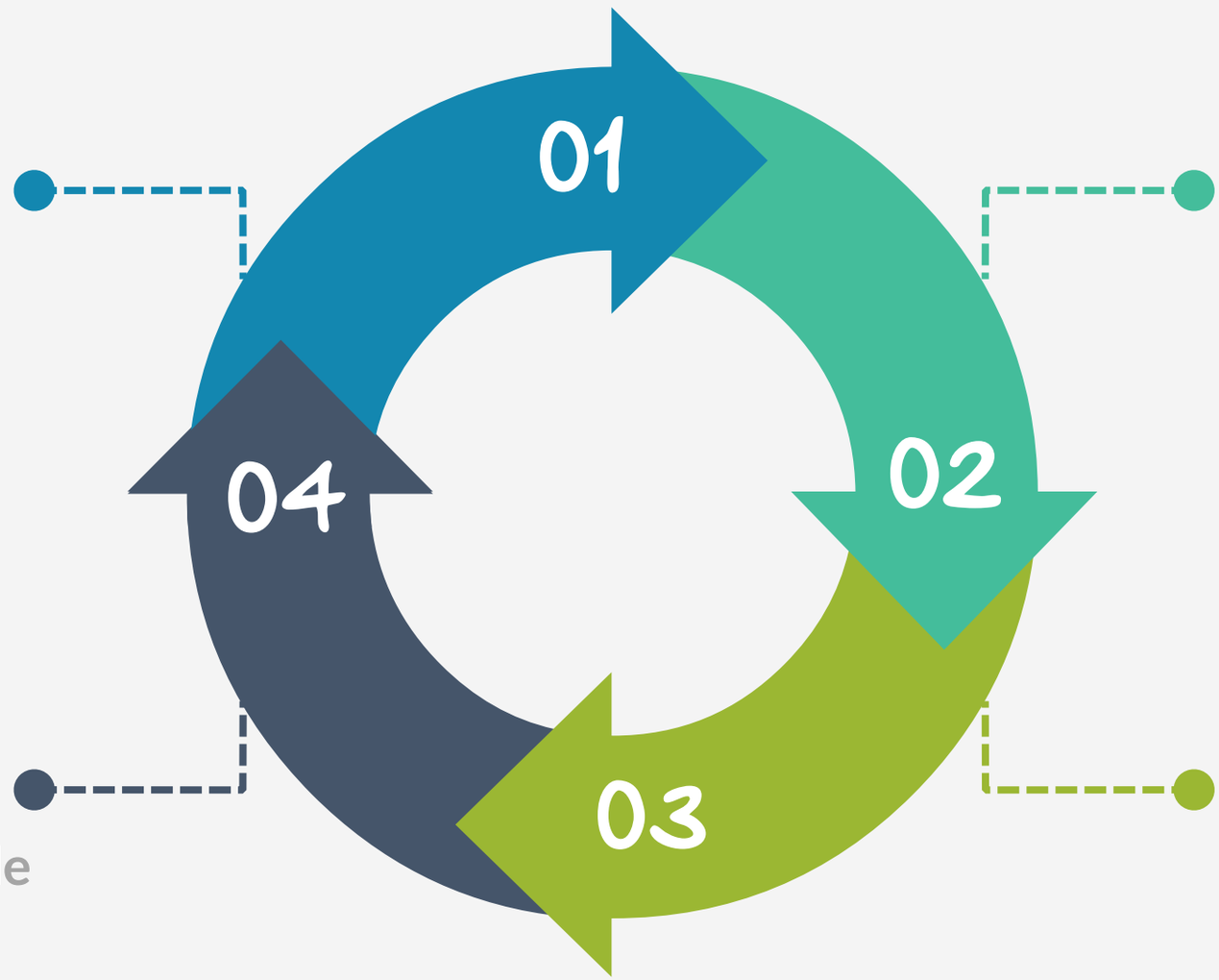
04 Non-functional concerns (offline data, latency,…)

05 Integrated UI and database templating

A TIER SPLITTING APPROACH IN A MAINSTREAM LANGUAGE

TIER SPLITTING

tierless js
javascript code
with annotations

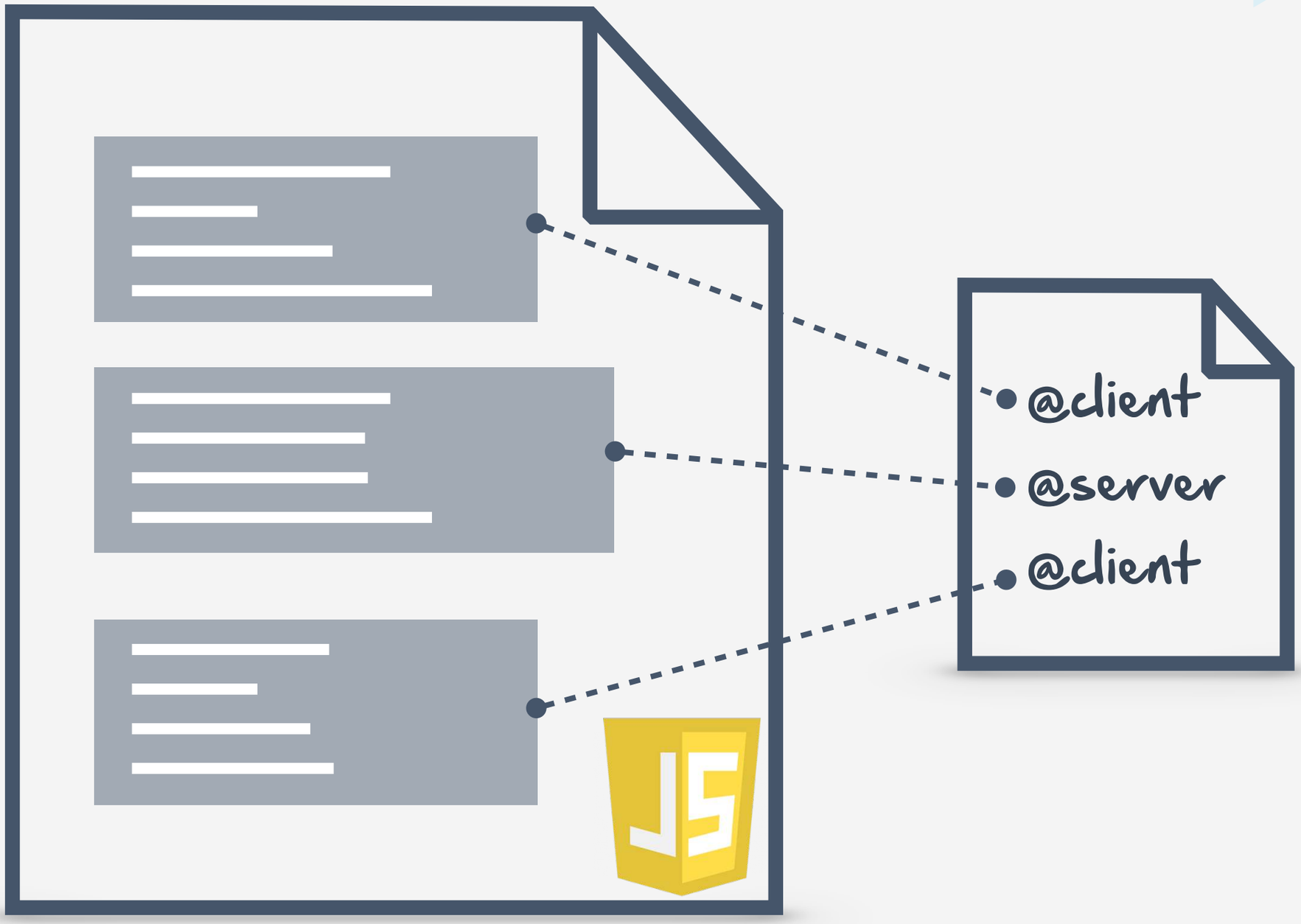


code analysis
tailored to javascript
for data and call
references

refactoring
refactor the split code
to distributed code

tier splitting
program slice the
server and client code

JAVASCRIPT



JAVASCRIPT

```
1  /* @server */
2  {
3    function broadcast(msg) {
4      displayMessage(msg);
5    }
6  }
7
8  /* @client */
9  {
10   var clientId = "user" + Math.random();
11
12   function displayMessage(msg) {
13     $("#msgs").append(msg);
14   }
15
16   broadcast(clientId + " says hello!");
17
18 }
```

three.js

HIGHCHARTS

jQuery
write less. do more.

Moment.js

mocha

JS Hint

WS

JSLint
The JavaScript Code Quality Tool

ANNOTATIONS

PLACEMENT	COMMUNICATION	DATA SHARING	FAILURE HANDLING
@	@	@	@
client	remoteCall	local	defineHandler
server	remoteFunction	copy	useHandler
shared	reply	observable	
ui	broadcast	replicated	
	blocking		
block level	**call / function level**	**declaration level**	**block / call level**

ANNOTATIONS

```
1  /* @server */
2  {
3     /* @observable */
4     var scoreCollection = [];
5
6     /* @remoteFunction */
7     function addScore(user, score) {
8      scoreCollection.push({user: user, score: score})
9     }
10 }
11
12 /* @client */
13 {
14   var user = "user" + Math.random();
15   /* @remoteCall */
16   addScore(user, 0);
17   scoreCollection.forEach(function (score) {
18     // display user - score
19   })
20 }
```

ANNOTATIONS

```
1  /* @server @useHandler: Retry(2), Buffer, Log */
2  {
3    /* @remoteFunction */
4    function shareOnTwitter(account, article) {
5      // call twitter API
6      /* @reply */
7      displayPopover("Article shared on twitter");
8
9    }
10 }
11
12 /* @client */
13 {
14   /* @remoteFunction */
15   function displayPopover (msg) {
16     $("#popover").text(msg);
17     $("#popover").show();
18   }
19 }
```

ANNOTATIONS

PLACEMENT	COMMUNICATION	DATA SHARING	FAILURE HANDLING
@	@	@	@
client	remoteCall	local	defineHandler
server	remoteFunction	copy	useHandler
shared	reply	observable	
ui	broadcast	replicated	
	blocking		
block level	**call / function level**	**declaration level**	**block / call level**

PROGRAM ANALYSIS

PROGRAM ANALYSIS

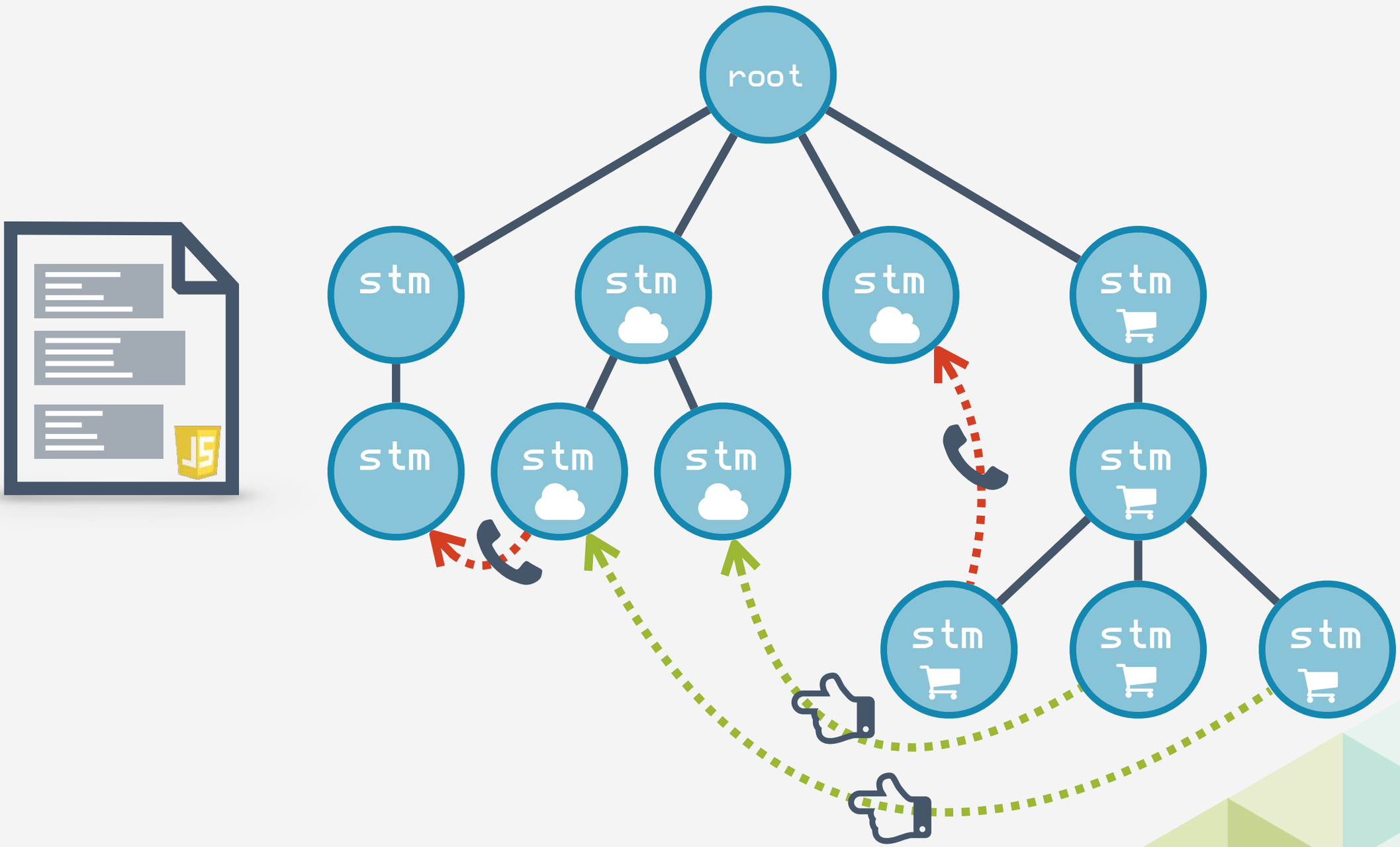
```
1 /* @server */
2 {
3   /* @remoteFunction */
4   function broadcast(msg) {
5     /* @remoteCall */
6     displayMessage(msg);
7   }
8 }
```

without analysis

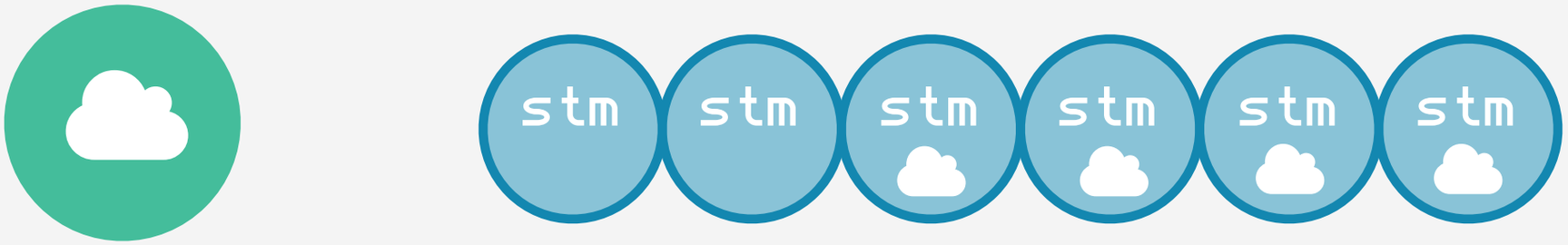
with analysis

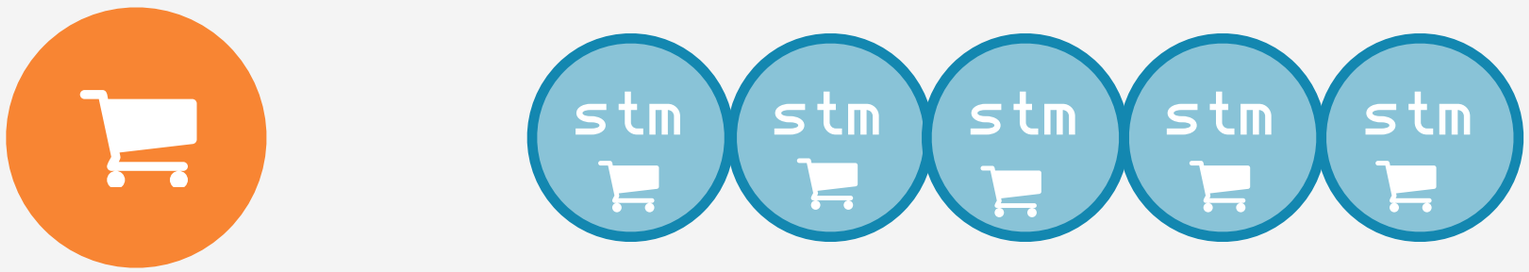
```
1 /* @server */
2 {
3   function broadcast(msg) {
4     displayMessage(msg);
5   }
6 }
```

PROGRAM DEPENDENCE GRAPH



TIER SPLITTING





Mark Weiser. 1981. **Program slicing**. In Proceedings of the 5th international conference on Software engineering (ICSE '81). IEEE Press, Piscataway, NJ, USA, 439-449.

CODE TRANSFORMING


```
1  /* SERVER CODE */
2
3  server.expose({
4    broadcast : function (msg, cb) {
5      server.rpc('displayMessage', msg);
6    }
7  })
8
9
10 /* CLIENT CODE */
11
12 var clientId = "user" + Math.random();
13 client.expose({
14   displayMessage : function (msg, cb) {
15     $("#msgs").append(msg);
16   }
17 })
18
19 client.rpc('broadcast',
20   clientId + " says hello!");
```

01 Tool support

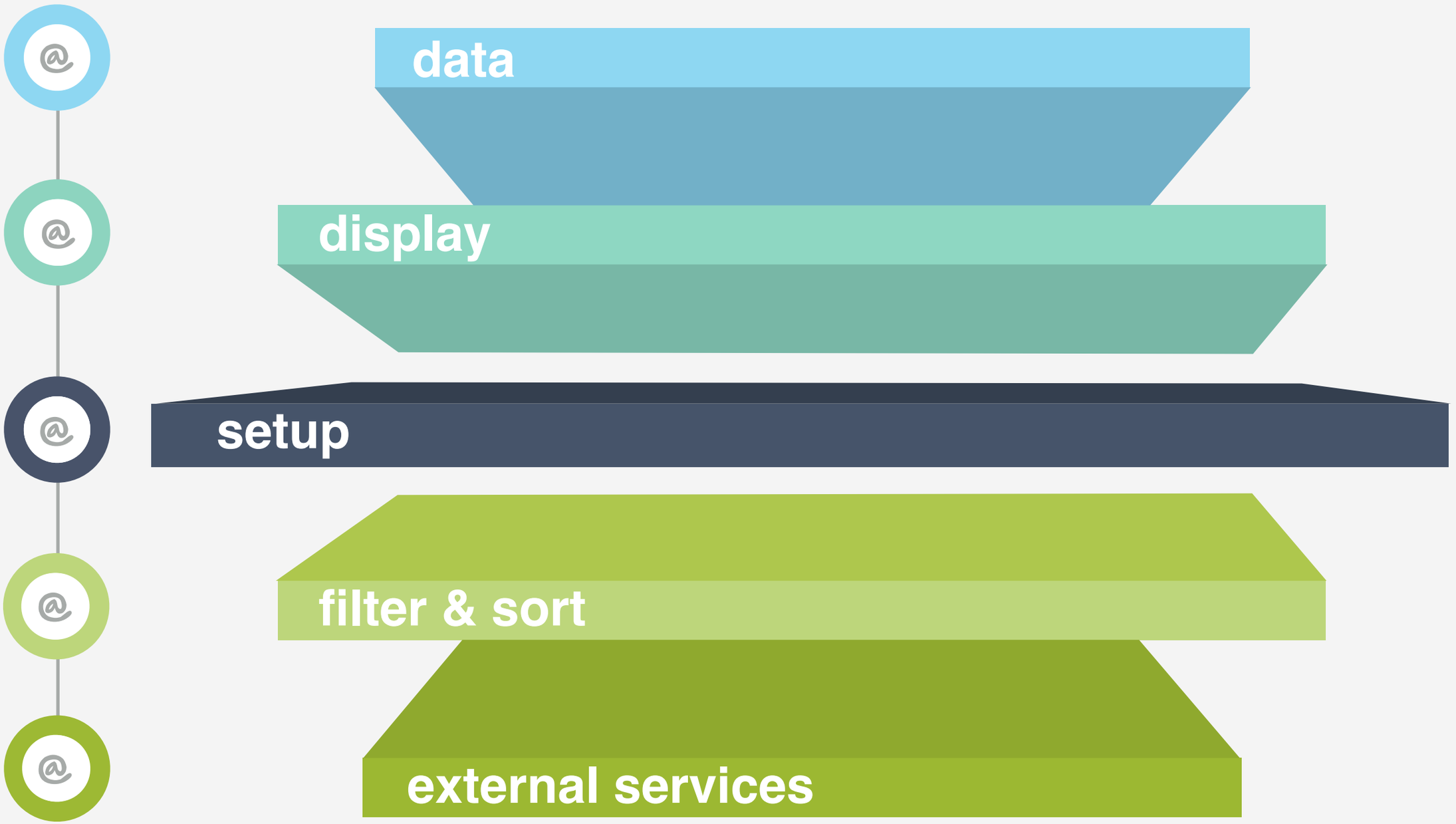
02 Mainstream language

03 Full-fledged JS on the client

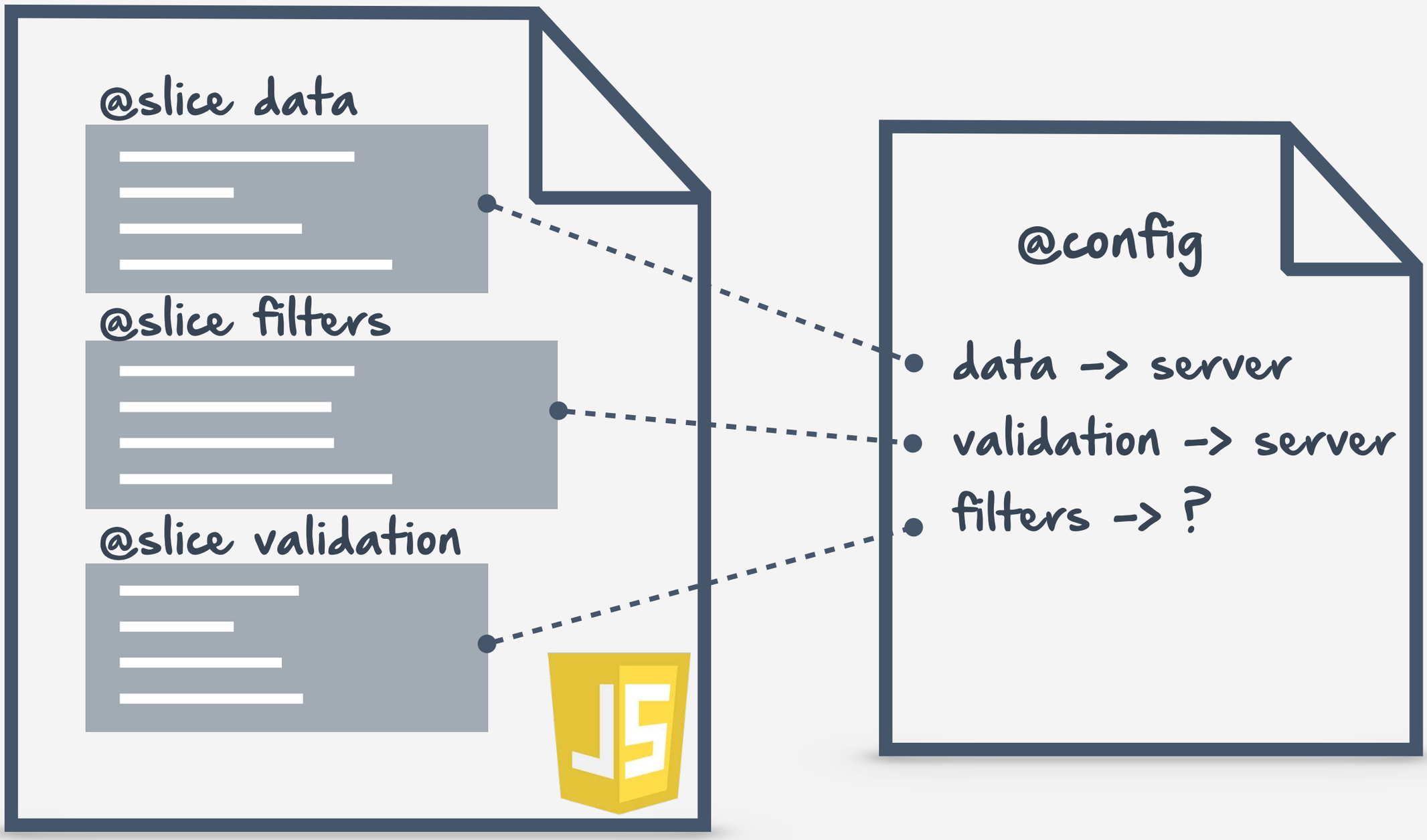
Non-functional concerns (offline data, latency,…) 04

Integrated UI and database templating 05

WEB SLICES



WEB SLICES CONFIGURATION



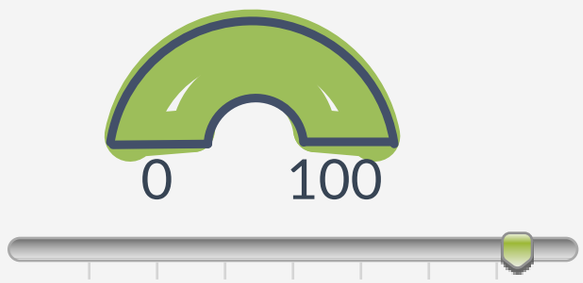
```
/* @slice users */
{
    var users = [];
    function addUser (id, name) {}
    function generateId () {}
}

/* @slice messages */
{
    var messages = [];
    function addMessage (user, msg) {}
}

/* @slice listeners */
{
    var currentUser;
    function sendListener () {}
    function addUserListener () {}
    $("#sendBtn").click(sendListener);
    $("#addUserListener").click(addUserListener);

}

/* @slice display */
{
    function displayMessages () {}
}
```

```
/* @config
      users : server,
      display : client,
      listeners : client
*/
```

PLACEMENT STRATEGIES

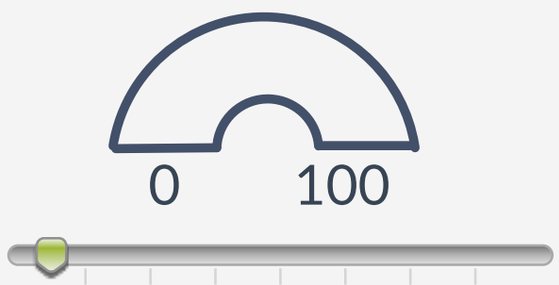
filters -> ?

offline availability



latency

security



01 Tool support

02 Mainstream language

03 Full-fledged JS on the client

Non-functional concerns (offline data, latency,…) 04

Integrated UI and database templating 05

STIP.JS

bit.ly/stipjs

Your JavaScript code:

Snippets

```
/* @config
     users : server,
     display : client,
     listeners : client

   @slice users */
{
    var users = [];

    function addUser (id, name) {
        var user = {id: id, name: name}
        users.push(user);
        return user;
    }

    function generateId () {
      var nr = Math.random();
      return nr;
    }
}

/* @slice messages */
{
    var messages = [];
    function addMessage (user, msg) {
```

Results

client | server | setup

Code

Offline report

Web slice distribution

CLIENT | SERVER

Detailed report

listeners client

DATA DEPENDENCIES

> Client: 0, > Server: 0

CALL DEPENDENCIES

> Client: 1, > Server: 2

Data: 0 100

Call: 33 (0 100)

display client

DATA DEPENDENCIES

> Client: 2, > Server: 0

CALL DEPENDENCIES

Data: 100 (0 100)

Call: 0 100

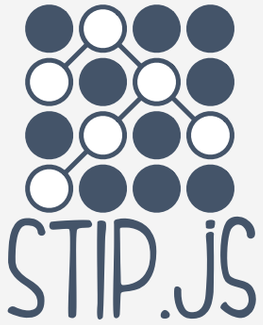
DISTRIBUTED PROGRAMMING

DISTRIBUTED PROGRAMMING

HOW WEB PROGRAMMING IS MORE THAN A SERVER AND SOME CLIENTS

TIERLESS APPROACHES

TIER SPLITTING FOR JS



WEB SLICES

PLACEMENT STRATEGIES

@laurephilips

lphilips@vub.ac.be

bit.ly/stipjs

Cracked Earth, Abraham Lake, photo by Mike Isaak Photography